Observability: Spring AI emphasises observability to help developers monitor and debug AI workflows. The Spring Boot Starter Actuator provides metrics such as token usage, model used, etc, and supports integration with external monitoring systems such as Zipkin, Splunk and DataDog, thus enabling visibility into model behaviour and system performance.

Chat Memory: Chat Memory allows Spring AI applications to maintain conversational context across interactions. The PromptChatMemoryAdvisor manages pre- and post- requests to the model, which makes this feature essential for building agentic AI systems that require continuity and coherence in multi-turn dialogues. Chat Memory can be configured to be used in-memory or can store history in a database.

Retrieval augmented generation with vector stores: Spring AI uses an advisor called QuestionAnswerAdvisor so the ChatClient knows it must consult the vector store for supporting documents before sending the request to the model.

Structured output: Spring AI can be configured such that the model map returns data to a strongly typed object, which can return the model response in JSON for programmatic access.

Local function tools: Spring AI can annotate any local function with @Tool and @ToolParam with descriptions in human language so that it can be invoked by an LLM as needed. This is a great re-use of existing libraries within a genAI application

Model Context Protocol: Model Context Protocol (MCP) is a standardised protocol that enables AI models to interact with external tools and resources in a structured way, regardless of the language in which they were written.

There are three flavours of MCP that Spring AI supports: STDIO, servlet based SSE, and WebFlux or WebMVC based HTTP streaming. You can easily configure the MCP server by adding the MCP information in application.properties.

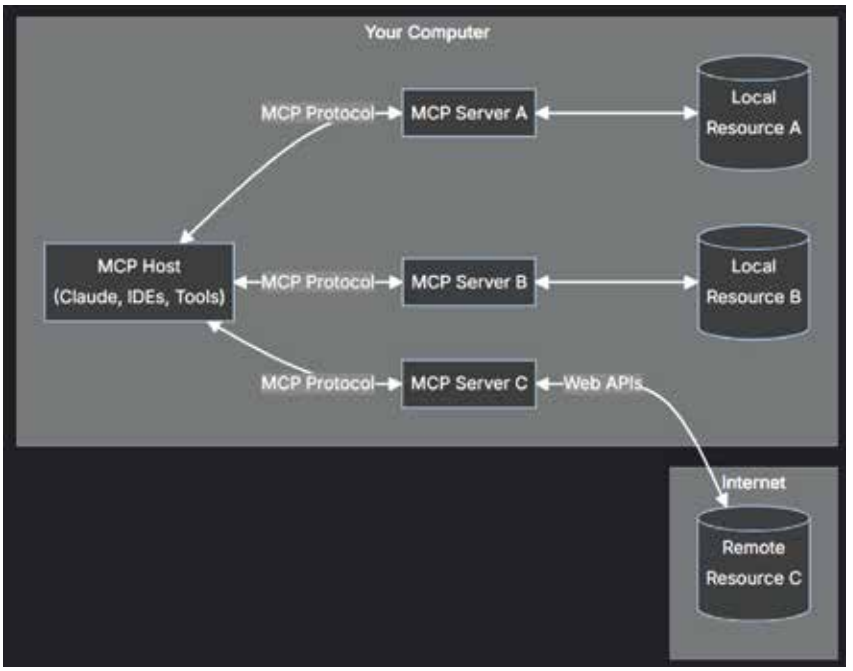

Figure 1: Model Context Protocol

Spring AI provides client and server starters to facilitate MCP integration. One interesting capability of MCP is its ability to dynamically update available tools at runtime. MCP servers can add or remove tools without restarting, MCP clients can detect these changes, and AI models can immediately use the new capabilities.

Building new age web applications with Spring AI

Spring AI's integration with Java Spring Boot provides a powerful combination for developing modern web applications across various industries.

Banking: In the banking sector, Spring AI can be employed to enhance customer service through intelligent chatbots, streamline loan approval processes with automated document analysis, and improve fraud detection using predictive models.

Insurance: Insurance companies can leverage Spring AI to automate claims processing, generate personalised policy recommendations, and provide virtual assistants that guide customers through complex insurance products.

Retail: Retailers can utilise Spring AI to create personalised shopping experiences, automate inventory management with predictive analytics, and develop visual search engines that allow customers to find products using images.

Healthcare: In healthcare, Spring AI can be used to develop virtual health assistants, automate medical transcription, and implement predictive models that aid in disease diagnosis and treatment planning.

Wealth management: Wealth management firms can employ Spring AI to generate personalised investment strategies, create voice-based financial reports, and develop intelligent advisory systems that assist clients in making informed financial decisions.

The challenges

Spring AI incorporates generative AI within the Spring ecosystem. Integrating advanced features like retrieval augmented generation, tool calling, and